Tori Overholtzer

CYB2200 Assignment – Static analysis tools

Objective: Practice using static analysis tools to analyze programs.

Due: Friday, APR 24, 2026, 11:59 pm on CANVAS.

Turn in: This report

Points: 60 points

Requirements:

- Generate ONE piece of C code that has 10 different vulnerabilities.
- Run 3 different static analysis tools to analyze the vulnerable code.
- While running the tools, finish the following report and the final summary.

Step 1: generate a piece of C code that includes each of these vulnerabilities.

You are allowed to use AI/LLM to generate vulnerable code for this step, but you MUST verify each of the vulnerabilities and understand why it is a vulnerability in the generated code.

Uninitialized variable
Buffer overflow
Unsafe function (caught the use of strcpy() or strcat())
Type mismatch
Format string vulnerability
Integer overflow
Off-by-one
Double free
Null pointer dereference
Use-after-free

(10 pts) Question: (A) If you used AI to generate this piece of C code, what was your experience with AI during the process? What was easy vs. what was difficult? (B) If you didn't use AI, then what was your experience generating this code yourself?

I used Claude to generate a vulnerable fortune teller program then verified the results. Overall, the experience wasn't bad. I did use a detailed prompt to get the input desired; a less detailed prompt would probably have produced less useful results. The easy part of generating code was the speed at which the code was generated. The more difficult part of generating code was reading, understanding and testing the code to the extent that it could be verified that the code was behaving as intended with the expected vulnerability examples.
If I wrote this code without using AI, it would have taken more time, but it would have been easier to

understand because I would have introduced each of these errors by implementing code piece by piece.

The prompt I used to generate the code was:

"Generate a vulnerable C code fortune teller that has each of the vulnerabilities: an uninitialized variable, a buffer overflow, an unsafe function such as the use of strcpy() or strcat(), a type mismatch, a format string vulnerability, an integer overflow, an off by one error, double free, a null pointer dereference and a use after free. Comment the vulnerabilities and generate a vulnerability summary."

In summary:

For the Uninitialized Variable vulnerability, it declares user_choice but never initializes this variable before use located in main();
For the Buffer Overflow vulnerability, it sets name field to 16 bytes but allows input to exceed this buffer size in the create_profile() function;
For the Unsafe Function vulnerability, it uses strcpy() and strcat() without bounds checking in the create_profile() function and the print_fortune_banner() function;
For the Type Mismatch vulnerability, it casts a char* return of strcpy to int in the create_profile() function.
For the Format String vulnerability, it uses user input passed directly to printf() without checking format in the print_fortune_banner() function.
For the Integer Overflow vulnerability, it can multiply large integers causing an overflow in the calculate_lucky_number() function.
For the Off-by-One vulnerability, % (max + 1) allows the index to equal to array size in the get_fortune_index() function.
For the Double Free vulnerability, the same pointer is freed twice in main() and in demonstrate_memory_bugs().
For the Null Pointer Deference vulnerability, it accesses a member of Null pointer in main().
For the Use After Free vulnerability, it accesses memory after free() function in demonstrate_memory_bugs() function.

(10 pts) Submit the C code.

See Attached File

Step 2: Run 3 different static analysis tools to analyze a vulnerable program generated from step 1.

Tori Overholtzer

Pick 3 static analysis tools presented in class:

- Clang/clang++ static analyzer
- Gcc static analyzer
- Cppcheck
- Infer

STATIC ANALYSIS TOOL #1

(10 pts) Run static analysis tool #1 **Clang/Clang++ Static Analyzer** to analyze the vulnerable programs. The result from the tool is (take a screenshot of the results):

Tori Overholtzer

```
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ ls
a.out  FortuneTellerVulnerable.c  FortuneTellerVulnerable.plist  output.txt
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ scan-build clang --analyze FortuneTellerVulnerable.c
 > output.txt
FortuneTellerVulnerable.c:31:5: warning: Value stored to 'result' is never read [deadcode.DeadStores]
   31 |     result = (int)strcpy(profile->name, name);  // Type mismatch in cast
      |     ^        ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
FortuneTellerVulnerable.c:89:5: warning: Attempt to free released memory [unix.Malloc]
   89 |     free(temp);
      |     ^~~~~~~~~~
FortuneTellerVulnerable.c:139:25: warning: The left operand of '==' is a garbage value [core.UndefinedBinaryOperatorResult]
  139 |         if (user_choice == 42) {
      |             ~~~~~~~~~~~ ^
3 warnings generated.
FortuneTellerVulnerable.c:31:5: warning: Value stored to 'result' is never read [deadcode.DeadStores]
   31 |     result = (int)strcpy(profile->name, name);  // Type mismatch in cast
      |     ^        ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
FortuneTellerVulnerable.c:89:5: warning: Attempt to free released memory [unix.Malloc]
   89 |     free(temp);
      |     ^~~~~~~~~~
FortuneTellerVulnerable.c:139:25: warning: The left operand of '==' is a garbage value [core.UndefinedBinaryOperatorResult]
  139 |         if (user_choice == 42) {
      |             ~~~~~~~~~~~ ^
3 warnings generated.
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$
```

```
  GNU nano 7.2                                          output.txt
scan-build: Using '/usr/lib/llvm-18/bin/clang' for static analysis
scan-build: Analysis run complete.
scan-build: 3 bugs found.
scan-build: Run 'scan-view /tmp/scan-build-2026-04-24-113030-5823-1' to examine bug reports.
```

Documents - scan-build results

User:	ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox
Working Directory:	/home/ubuntu-semantic-analyzers/Documents
Command Line:	clang --analyze FortuneTellerVulnerable.c
Clang Version:	Ubuntu clang version 18.1.3 (1ubuntu1)
Date:	Fri Apr 24 11:30:30 2026

Bug Summary

Bug Type	Quantity	Display?
All Bugs	**3**	☑
Logic error		
Result of operation is garbage or undefined	1	☑
Memory error		
Double free	1	☑
Unused code		
Dead assignment	1	☑

Tori Overholtzer

Reports

Bug Group	Bug Type ▾	File	Function/Method	Line	Path Length			
Unused code	Dead assignment	FortuneTellerVulnerable.c	create_profile	31	1	View Report	Report Bug	Open File
Memory error	Double free	FortuneTellerVulnerable.c	demonstrate_memory_bugs	89	10	View Report	Report Bug	Open File
Logic error	Result of operation is garbage or undefined	FortuneTellerVulnerable.c	main	139	6	View Report	Report Bug	Open File

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <limits.h>

#define MAX_FORTUNES 5
#define SMALL_BUFFER 16

const char *fortunes[] = {
    "You will find great wealth!",
    "Love is on the horizon.",
    "A journey awaits you.",
    "Beware of false friends.",
    "Success is within reach!"
};

typedef struct {
    char name[SMALL_BUFFER];
    int lucky_number;
    char *personal_fortune;
} UserProfile;

// VULNERABILITY 9: Null pointer dereference setup
UserProfile *create_profile(const char *name) {
    UserProfile *profile = (UserProfile *)malloc(sizeof(UserProfile));

    // VULNERABILITY 4: Type mismatch - strcpy expects char*, returns char*
    // but we're ignoring return and using int-sized variable conceptually
    int result;
    result = (int)strcpy(profile->name, name);  // Type mismatch in cast

    Value stored to 'result' is never read

    // VULNERABILITY 3: Unsafe function - strcpy with no bounds checking
    // Also causes VULNERABILITY 2: Buffer overflow if name > 15 chars
```

Tori Overholtzer

```c
}

void demonstrate_memory_bugs(void) {
    char *temp = (char *)malloc(32);

                    7 ← Memory is allocated →

    strcpy(temp, "Temporary fortune");

    printf("Temp: %s\n", temp);

    free(temp);

    8 ← Memory is released →

    // VULNERABILITY 8: Double free
    free(temp);

    9 ← Attempt to free released memory

    // VULNERABILITY 10: Use after free
    printf("After free: %s\n", temp);
}

int main(int argc, char *argv[]) {
    // VULNERABILITY 1: Uninitialized variable
    int user_choice;  // Never initialized before potential use
```

```
127     // VULNERABILITY 7: Off-by-one - may access fortunes[5] which doesn't exist
128     int fortune_idx = get_fortune_index(MAX_FORTUNES);
129
130     // Potential crash here due to off-by-one
131     if (fortune_idx < MAX_FORTUNES + 1) {  // Wrong bounds check
            2 ← Assuming the condition is true →
        3 ← Taking true branch →
132         printf("\nðŸ”® Your fortune: %s\n", fortunes[fortune_idx]);
133     }
134
135     // VULNERABILITY 1: Using uninitialized user_choice
136     // This path might execute with garbage value
137     if (argc > 1) {
            4 ← Assuming 'argc' is > 1 →
        5 ← Taking true branch →
138         // user_choice was never set but we use it
139         if (user_choice == 42) {
                    6 ← The left operand of '==' is a garbage value
140             printf("Secret mode activated!\n");
141         }
142     }
143
144     // VULNERABILITY 9: Null pointer dereference
145     printf("\nAttempting to access null user profile...\n");
146     // This will crash - dereferencing NULL
147     // printf("Null user name: %s\n", null_user->name);
148     // Uncomment above to trigger crash
```

The result of running the Clang Static Analyzer is that the analyzer found three types of bugs: unused code (value of result is never used) , a memory error (double free) and a logic error (result is garbage value because user_choice is uninitialized). It also presented some warnings: type mismatch in cast around assigning the value of result, use after free, undefined binary operator.

Question: What is the tool good for? Not good for? Does it give you false positive results? Keep track of what vulnerabilities (Out of the 10) were detected.

The Clang Static Analyzer tool is good for many debugging tasks. It can detect issues with memory management such as memory leaks, use after free errors, and double free vulnerabilities. It can also detect pointer and logical errors such as null pointer dereferences, division by zero, and uninitialized variable reads. It can detect API misuse such as incorrect usage of standard C/C++ libraries. It can also find complex low level security flaws such as tainted data propagation.

Tori Overholtzer

The Clang Static Analyzer is not good at doing Multithreaded Code Analysis because it creates separate graphs for each thread, preventing the sharing of program state. It often fails to recognize thread creation events, which prevents it from detecting concurrency bugs like race conditions. It is also not great at doing large projects within tight developing windows because it is significantly slower than standard compilation; it must explore numerous possible program paths. This can make it too expensive for frequent use in high-speed pipelines. This analyzer is also not good at being a zero false positive environment. This tool is prone to path explosion where it explores practically impossible runtime paths leading to false positive reports. It is also not great at Inter-Procedural Analysis because it uses single translation units by default and while cross translation unit support exists, it can be complex to configure. The analyzer also cannot catch specific runtime issues since it does not execute code. It cannot find errors that only manifest under specific runtime configurations, library environments, or external system interactions. It is also not good at finding high-level security flaws such as high-level architectural security issues like authentication flaws, access control problems, or insecure cryptography usage.

The Clang Static Analzyer has been known to give false positive results; however, in the running of the fortune teller program, it didn't produce any false positive results this time.

To see what vulnerabilities were detected in the Fortune Teller program, see the chart located in the summary section of this document.

STATIC ANALYSIS TOOL #2

(10 pts) Run static analysis tool #2 **GCC Static Analyzer** to analyze the vulnerable program. The result from the tool is (take a screenshot):

Tori Overholtzer

```
buntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ ls
.out  FortuneTellerVulnerable.c
buntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ gcc -fanalyzer FortuneTellerVulnerable.c
ortuneTellerVulnerable.c: In function ‘create_profile’:
ortuneTellerVulnerable.c:31:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
   31 |     result = (int)strcpy(profile->name, name);  // Type mismatch in cast
      |              ^
ortuneTellerVulnerable.c: In function ‘print_fortune_banner’:
ortuneTellerVulnerable.c:63:5: warning: format not a string literal and no format arguments [-Wformat-security]
   63 |     printf(user_input);
      |     ^~~~~~
ortuneTellerVulnerable.c: In function ‘create_profile’:
ortuneTellerVulnerable.c:37:31: warning: dereference of possibly-NULL ‘profile’ [CWE-690] [-Wanalyzer-possible-null-dereference]
   37 |     profile->personal_fortune = NULL;
      |                               ^
  ‘create_profile’: event 1
    |
    |   26 |     UserProfile *profile = (UserProfile *)malloc(sizeof(UserProfile));
    |      |                                           ^~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |                                           |
    |      |                                           (1) this call could return NULL
    |
  ‘create_profile’: event 2
    |
    |   37 |     profile->personal_fortune = NULL;
    |      |                               ^
    |      |                               |
    |      |                               (2) ‘profile’ could be NULL: unchecked value from (1)
    |
```

```
FortuneTellerVulnerable.c: In function ‘set_personal_fortune’:
FortuneTellerVulnerable.c:43:5: warning: use of possibly-NULL ‘*profile.personal_fortune’ where non-null expected [CWE-690] [-Wanalyzer-possible-null-argument]
   43 |     strcpy(profile->personal_fortune, fortune);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  ‘set_personal_fortune’: events 1-2
    |
    |   42 |     profile->personal_fortune = (char *)malloc(strlen(fortune) + 1);
    |      |                                           ^~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |                                           |
    |      |                                           (1) this call could return NULL
    |   43 |     strcpy(profile->personal_fortune, fortune);
    |      |     ~~~~~~~~~~~~~~~~~~~
```

Tori Overholtzer

```
FortuneTellerVulnerable.c: In function 'free_profile':
FortuneTellerVulnerable.c:47:17: warning: use after 'free' of 'profile' [CWE-416] [-Wanalyzer-use-after-free]
   47 |     free(profile->personal_fortune);
      |          ~~~~~~~^~~~~~~~~~~~~~~~~~
  'main': events 1-2
    |
    |   95 | int main(int argc, char *argv[]) {
    |      |     ^~~~
    |      |     |
    |      |     (1) entry to 'main'
    |......
    |  114 |     current_user = create_profile(name_buffer);
    |      |                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |                    |
    |      |                    (2) calling 'create_profile' from 'main'
    |
    +--> 'create_profile': events 3-4
           |
           |   25 | UserProfile *create_profile(const char *name) {
           |      |              ^~~~~~~~~~~~~~
           |      |              |
           |      |              (3) entry to 'create_profile'
           |   26 |     UserProfile *profile = (UserProfile *)malloc(sizeof(UserProfile));
           |      |                                           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~
           |      |                                           |
           |      |                                           (4) allocated here
           |
         'create_profile': event 5
           |
           |   37 |     profile->personal_fortune = NULL;
           |      |                               ^
           |      |                               |
           |      |                               (5) assuming 'profile' is non-NULL
           |
    <------+
    |
  'main': events 6-11
    |
    |  114 |     current_user = create_profile(name_buffer);
    |      |                    ^~~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |                    |
    |      |                    (6) returning to 'main' from 'create_profile'
    |......
    |  137 |     if (argc > 1) {
    |      |        ~
    |      |        |
    |      |        (7) following 'false' branch (when 'argc <= 1')...
    |......
    |  145 |     printf("\nAttempting to access null user profile...\n");
    |      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |     |
    |      |     (8) ...to here
    |......
    |  151 |     if (null_user != NULL) {
    |      |        ~
    |      |        |
    |      |        (9) following 'false' branch (when 'null_user' is NULL)...
    |......
    |  156 |         char *bad_access = null_user->name;  // Compiles but crashes at runtime
```

Tori Overholtzer

```
  |------
  |  151 |     if (null_user != NULL) {
  |      |        ~
  |      |        |
  |      |        (9) following 'false' branch (when 'null_user' is NULL)...
  |------
  |  156 |         char *bad_access = null_user->name;  // Compiles but crashes at runtine
  |      |               ~~~~~~~~~~~
  |      |               |
  |      |               (10) ...to here
  |------
  |  164 |     free_profile(current_user);
  |      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~
  |      |     |
  |      |     (11) calling 'free_profile' from 'main'
  |
  +--> 'free_profile': events 12-13
       |
       |   46 | void free_profile(UserProfile *profile) {
       |      |      ^~~~~~~~~~~~
       |      |      |
       |      |      (12) entry to 'free_profile'
       |   47 |     free(profile->personal_fortune);
       |   48 |     free(profile);
       |      |     ~~~~~~~~~~~~~
       |      |     |
       |      |     (13) freed here
       |
  <-------+
  |
'main': events 14-15
  |
  |  164 |     free_profile(current_user);
  |      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~
  |      |     |
  |      |     (14) returning to 'main' from 'free_profile'
  |------
  |  169 |     free_profile(current_user);  // Double free
  |      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~
  |      |     |
  |      |     (15) calling 'free_profile' from 'main'
  |
  +--> 'free_profile': events 16-17
       |
       |   46 | void free_profile(UserProfile *profile) {
       |      |      ^~~~~~~~~~~~
       |      |      |
       |      |      (16) entry to 'free_profile'
       |   47 |     free(profile->personal_fortune);
       |      |          ~~~~~~~~~~~~~~~~~~~~~~~~~~
       |      |                  |
       |      |                  (17) use after 'free' of 'profile'; freed at (13)
       |
```

Tori Overholtzer

```
FortuneTellerVulnerable.c: In function 'demonstrate_memory_bugs':
FortuneTellerVulnerable.c:82:5: warning: use of possibly-NULL 'temp' where non-null expected [CWE-690] [-Wanalyzer-possible-null-argument]
   82 |     strcpy(temp, "Temporary fortune");
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  'demonstrate_memory_bugs': events 1-2
    |
    |   81 |     char *temp = (char *)malloc(32);
    |      |                          ^~~~~~~~~
    |      |                          |
    |      |                          (1) this call could return NULL
    |   82 |     strcpy(temp, "Temporary fortune");
    |      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |     |
    |      |     (2) argument 1 ('temp') from (1) could be NULL where non-null expected
    |
```

```
<built-in>: note: argument 1 of '__builtin_memcpy' must be non-null
FortuneTellerVulnerable.c:89:5: warning: double-'free' of 'temp' [CWE-415] [-Wanalyzer-double-free]
   89 |     free(temp);
      |     ^~~~~~~~~~
  'demonstrate_memory_bugs': events 1-4
    |
    |   81 |     char *temp = (char *)malloc(32);
    |      |                          ^~~~~~~~~
    |      |                          |
    |      |                          (1) allocated here
    |   82 |     strcpy(temp, "Temporary fortune");
    |      |     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    |      |     |
    |      |     (2) assuming 'temp' is non-NULL
    |......
    |   86 |     free(temp);
    |      |     ~~~~~~~~~~
    |      |     |
    |      |     (3) first 'free' here
    |......
    |   89 |     free(temp);
    |      |     ~~~~~~~~~~
    |      |     |
    |      |     (4) second 'free' here; first 'free' was at (3)
    |
```

```
FortuneTellerVulnerable.c: In function 'main':
FortuneTellerVulnerable.c:139:12: warning: use of uninitialized value 'user_choice' [CWE-457] [-Wanalyzer-use-of-uninitialized-value]
  139 |         if (user_choice == 42) {
      |            ^
  'main': events 1-5
    |
    |   97 |     int user_choice;  // Never initialized before potential use
    |      |         ^~~~~~~~~~~
    |      |         |
    |      |         (1) region created on stack here
    |      |         (2) capacity: 4 bytes
    |......
    |  137 |     if (argc > 1) {
    |      |        ~
    |      |        |
    |      |        (3) following 'true' branch (when 'argc > 1')...
    |  138 |         // user_choice was never set but we use it
    |  139 |         if (user_choice == 42) {
    |      |            ~
    |      |            |
    |      |            (4) ...to here
    |      |            (5) use of uninitialized value 'user_choice' here
    |
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$
```

The result of running the GCC Static Analyzer is that the analyzer found memory safety issues (double free error of variable temp, use after free error of profile and the null dereference) and logic errors (uninitalized value of user_choice, type mismatch during the cast for the value of result and the format string vulnerability with the use of unsafe formatting in printf().)

Question: What is the tool good for? Not good for? Does it give you false positive results? Keep track of what vulnerabilities (Out of the 10) were detected.

The GCC Static Analyzer tool is good for many debugging tasks. It can detect issues with memory management and is highly effective at identifying double free errors, memory leaks, and use after free vulnerabilities. It can also catch pointer issues such as null pointer dereferences and uses of uninitialized values by tracking state changes across function calls. Newer versions can also include a taint mode to detect attacker-controlled inputs used unsafely in sensitive operations like array indexing or memory allocation. It also handles resource handling other than memory resources such as double fclose calls on file streams.

The GCC Static Analyzer tool is not good for whole project analysis without a linker as it operates on a single translation unit at a time. It cannot see inside functions defined in other source files unless you use link time optimization which can significantly complicate the process. The GCC Static Analyzer is also not good when looking for fast compilation cycles because enabling –fanalyzer significantly increases compile time. This makes the analyzer unsuitable for every save and compile iteration during active development and is better suited for periodic deep scans. The GCC Static Analyzer doesn't perform well in low memory environments as it is extremely resource intensive. On large projects or when using link time optimization, it can consume massive amounts of memory causing builds to fail on standard development machines. It also has limited language support for non-c languages. It was primarily designed for and is most effective with C but has experimental support for cpp. It is not suitable for other languages like java or python. It can produce false positives because it explores many potential execution paths. It can also produce false negatives missing real bugs, especially for unprioritized warnings like tainted data and uninitialized values. The GCC Static Analyzer also is not easily extensible; generally, it is not easy to write your own project specific checks without potentially forking or deeply modifying the GCC source code. The GCC Static Analyzer cannot detect errors that only manifest during execution, such as performance bottlenecks, UI glitches, or logic errors dependent on external runtime data.

The GCC Static Analyzer has been known to give false positive results; however, in the running of the fortune teller program, it didn't produce any false positive results this time.

To see what vulnerabilities were detected in the Fortune Teller program, see the chart located in the summary section of this document.

STATIC ANALYSIS TOOL #3

(10 pts) Run static analysis tool #3 **CPPCheck** to analyze the vulnerable program. The result from the tool is:

Tori Overholtzer

```
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ cppcheck FortuneTellerVulnerable.c
Checking FortuneTellerVulnerable.c ...
FortuneTellerVulnerable.c:89:5: error: Memory pointed to by 'temp' is freed twice. [doubleFree]
    free(temp);
    ^
FortuneTellerVulnerable.c:86:5: note: Memory pointed to by 'temp' is freed twice.
    free(temp);
    ^
FortuneTellerVulnerable.c:89:5: note: Memory pointed to by 'temp' is freed twice.
    free(temp);
    ^
FortuneTellerVulnerable.c:156:28: error: Null pointer dereference: null_user [nullPointer]
        char *bad_access = null_user->name;  // Compiles but crashes at runtime
                           ^
FortuneTellerVulnerable.c:101:30: note: Assignment 'null_user=NULL', assigned value is 0
    UserProfile *null_user = NULL;  // For null pointer demo
                             ^
FortuneTellerVulnerable.c:156:28: note: Null pointer dereference
        char *bad_access = null_user->name;  // Compiles but crashes at runtime
                           ^
FortuneTellerVulnerable.c:139:13: error: Uninitialized variable: user_choice [uninitvar]
        if (user_choice == 42) {
            ^
ubuntu-semantic-analyzers@ubuntu-semantic-analyzers-VirtualBox:~/Documents$ ls
```

The result of running CPPCheck is that the analyzer found memory safety issues (double free error of variable temp and the null dereference) and a logic error (uninitialized value of user_choice.)

Question: What is the tool good for? Not good for? Does it give you false positive results? Keep track of what vulnerabilities (Out of the 10) were detected.

CppCheck is good for memory management due to its ability to detect memory leaks, mismatched allocations, and mismatched deallocations. It is also good at finding logic and value errors such as uninitialized variables, out of bound array access, and integer overflows. It can also flag unused functions, redundant code, and variables with scopes that could be reduced. It can flag incorrect usage of standard template library functions and can display portability issues and issues with implementation defined behaviors.

CppCheck is not good for catching all possible bugs. It tries to avoid noise and is not good at finding every single bug in the code. It only flags high confidence errors, meaning it will miss subtle issues that more aggressive nosier analyzers might catch. It is also not good for semantic understanding. CPPCheck doesn't perform a strict parse of your code, so it can struggle with complex C grammar ambiguities such as distinguishing between multiplication and pointer dereferencing, leading to missed bugs or incorrect diagnostics. It doesn't perform Deep Path Sensitivity; it uses a lighter flow-sensitive analysis. While this is faster than other analysis, it may not track variables through extremely complex logic branches as effectively as specialized formal verification tools. It may also need External Library Context; without manual library configuration files; CPPCheck is unaware of the behavior of external libraries. It may fail to detect memory leaks if an object is freed by a library specific parent-child relationship it doesn't recognize. CPPCheck also doesn't provide comprehensive CVE coverage. A study showed that it could only successfully identify a fraction of known security vulnerabilities.

Tori Overholtzer

CPPCheck can report false positives however it's designed around a philosophy to minimize them. It focuses on high certainty detection rather than catching every possible bug, but it is not immune to false warnings particularly with complex code or when using the –inconclusive flag. With the Fortune Teller program, it did not produce any false positives.

To see what vulnerabilities were detected in the Fortune Teller program, see the chart located in the summary section of this document.

Summary

(10 pts) Step 3: Final summary of using the three static analysis tools. Compare these 3 static analysis tools. For columns 2, 3, and 4, if the tool can identify the bug, mark "YES". Otherwise, mark "NO".

Regarding the Fortune Teller program, the GCC Static Analyzer performed the best at vulnerability detection based on the number of vulnerabilities found.

Vulnerability	Clang/Clang++ Static Analyzer	GCC Static Analyzer	CPPCheck
Uninitialized variable	Yes	Yes	Yes
Buffer overflow	No	No	No
Unsafe function (caught the use of strcpy() or strcat())	No	Yes	No
Type mismatch	Yes	Yes	No
Format string vulnerability	No	Yes	No
Integer overflow	No	No	No
Off-by-one	No	No	No
Double free	Yes	Yes	Yes
Null pointer dereference	No	Yes	Yes
Use-after-free	Yes	Yes	No
Number of detected vulnerabilities (with warnings)	4	7	3

Please submit this report and the C code to CANVAS.